

User's Guide for the Multiscale Full-field Solver for Electrochemical Systems

Kaiyu Li kaiyuli0225@gmail.com*

Abstract

In this report, we elaborate a multiscale full-field framework [1] for electrochemical systems governed by Nernst-Planck-Poisson equations. By integrating domain decomposition with a quasi-1D approximation, the framework efficiently resolves the layered structure above electrodes while significantly reducing computational costs and preserving solution accuracy. Though broadly applicable to diverse electrochemical systems, we validate the approach through a case study on water splitting. To enhance practical adoption, we detail two parallel implementation approaches: a user-friendly graphic-interface workflow and a code-based strategy. This dual approach ensures accessibility for both novice users and advanced developers seeking customization.

1 Methodology

1.1 Model formulation

While this multiscale full-field solver is applicable to various electrochemical systems, we take a hydrogen-evolving cathode surface in the potassium hydroxide solution as an example. The bulk solution is set to be 0.1M KOH saturated with H_2 ($C_{H_2,\infty} = 0.8\text{mM}$), in line with typical experimental working conditions[2]. In the water-splitting process, the chemical reaction on the cathode surface is:



the net rate of which is described by the Tafel equation when the kinetic overpotential, which is the potential difference between electrified surface and outer Helmholtz layer (OHL), η_s is large enough:

$$\eta_s = \eta_0 \log_{10}\left(\frac{j}{j_0}\right) \quad (2)$$

where j_0 and η_0 are the exchange current density and the Tafel slope, respectively, parameters dictated by electrode materials and temperature.[3] Our simulation employs $j_0 = 0.1\text{A}$ and $\eta_0 = 120\text{mV}$, values that align with those observed in previous experiments.[2] Based on the charge conservation, the molar flux of OH^- (k_{OH^-}) and H_2 (k_{H_2}) can be expressed as:

$$k_{OH^-} = 2k_{H_2} = \frac{j}{F} \quad (3)$$

where F is the Faraday constant. In the present study, we consider the steady-state transport of ions and gas, which follows:

$$\nabla \cdot J_i = 0, i = K^+, OH^-, H_2 \quad (4)$$

*This userguide is based on the solver developed by Zhang et al. [1]. The author greatly acknowledges the valuable guidance and constructive comments from Professor Lenan Zhang and Ms. Muhan Zhang.

where J_i is the molar flux of each species. With convection neglected, the transport of the ions is dictated by diffusion and migration, while the transport of gas is only dictated by diffusion. Therefore, J_i can be uniformly described by the following Nernst-Planck equation:

$$J_i = -D_i \nabla C_i - \frac{D_i C_i z_i F}{RT} \nabla \phi \quad (5)$$

Here, D_i , C_i , and z_i are the diffusivity, the molar concentration, and the charge of species i , respectively. R is the gas constant, and T is the temperature which we set as 25 °C. ϕ is the electrical potential that can be described by the Poisson-Boltzmann equation as follows:

$$\nabla \cdot (\epsilon_0 \epsilon_r \nabla \phi) = -F \sum z_i C_i \quad (6)$$

where ϵ_0 is the vacuum permittivity and ϵ_r is the relative permittivity of water. We assume $\phi = 0$ in the bulk, and calculate the potential gradient at the outer Helmholtz plane (OHP) $[\nabla \phi]_{OHP}$ using Eq. 2 and the following equation:

$$\eta_s = \phi_s - \phi_m = -\lambda_s \mathbf{n} \cdot [\nabla \phi]_{OHP} \quad (7)$$

where ϕ_s and ϕ_m are the electrical potential at the OHP and at the cathode surface, respectively. Key simulation parameters of the alkaline water electrolysis can be seen in Tab. 1.

Table 1: Parameters used in the simulation and their physical meanings.

Symbol	Value	Physical Meaning	Reference
$C_{K^+, \infty}$	0.1 M	Bulk K^+ molar concentration	1–4
$C_{OH^-, \infty}$	0.1 M	Bulk OH^- molar concentration	1–4
$C_{H_2, \infty}$	0.8 mM	Dissolved H_2 concentration in the bulk solution	5
λ_s	0.5 nm	Stern layer thickness	6,7
j_0	0.1 A/m ²	Exchange current density	8,9
η_0	120 mV	Tafel slope	8,9
T	25 °C	System temperature	–
ϵ_r	78.5	Relative permittivity of water	–
F	96485.3 C/mol	Faraday constant	–
D_{K^+}	2×10^{-9} m ² /s	K^+ diffusivity in water	7
D_{OH^-}	5×10^{-9} m ² /s	OH^- diffusivity in water	7
D_{H_2}	4.5×10^{-9} m ² /s	H_2 diffusivity in water	10
K_H	0.8 mM/atm	Henry’s solubility constant for H_2 in water	5

1.2 Computational framework

In the present study, we leverage a full-field multiscale framework[1] to numerically solve for the equations listed above. Although the electrolytic cell typically spans several centimeters in industrial applications, intricate features exist near its surface at length scales as small as a few nanometers. This brings huge challenges to the simulation of the electrochemical gas evolution process near the surface, as it is hard to find a mesh that is compatible with this span of length scale. A relatively coarse mesh that works efficiently with the centimeter scale may have a poor performance in capturing features at the nanometer length scale, while a fine mesh suitable for the

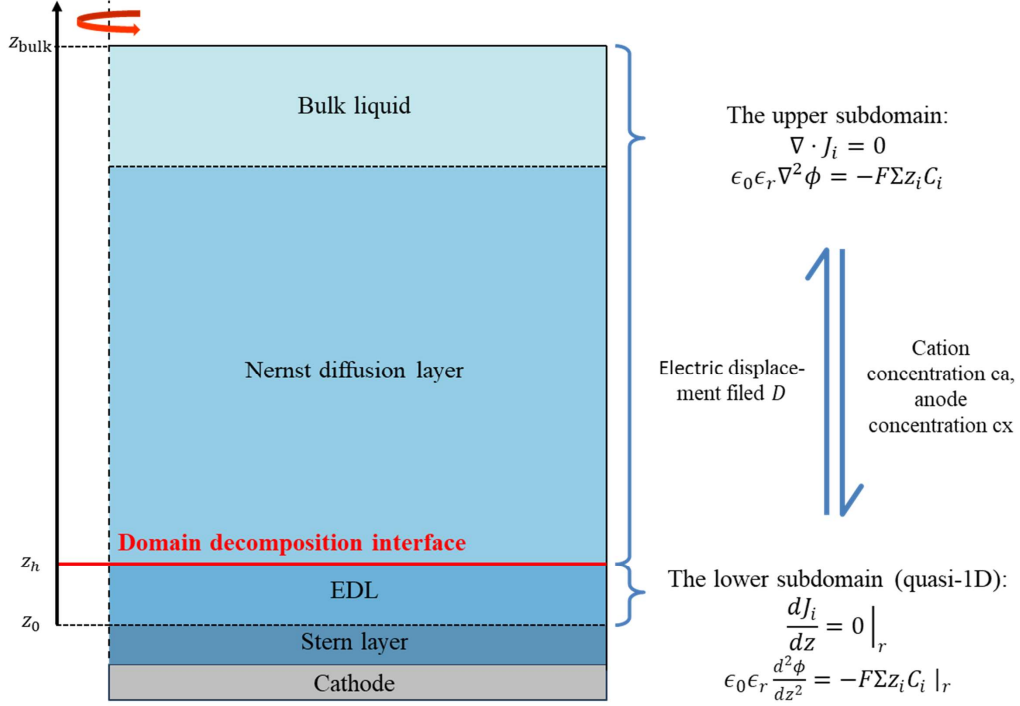


Figure 1: Schematic of the computational framework. The computational domain spans from the OHP ($z = z_0$) to the bulk solution ($z = z_{\text{bulk}}$), and is first geometrically decoupled at the interface of the EDL and the Nernst diffusion layer ($z = z_h$). The upper subdomain solved in COMSOL Multiphysics and the lower subdomain solved in MATLAB are coupled together to get a full-field solution, by using the MATLAB solver as a boundary condition in the COMSOL simulation. It uses the boundary concentration from the upper subdomain to calculate the boundary electric field in the lower subdomain, which is then converted to the electric displacement boundary condition D for the upper subdomain.

nanometer length scale comes with an unaffordably high computational cost when extended to the entire domain.

To address the aforementioned problem and strike a balance between the accuracy and the computational cost, the computational domain is decoupled into two non-overlapping subdomains at the interface of the electric double layer (EDL) and the Nernst diffusion layer¹, as demonstrated by the red line in Fig. 1. While the upper subdomain from the interface to the bulk is solved in COMSOL Multiphysics and the lower subdomain from the OHP to the top of the EDL is solved in MATLAB, they are physically coupled together to form a correct full-field solution.

For the lower subdomain from the OHP to the top of the EDL, where we have intricate characteristics at nanometer scale and large gradients, a quasi-1D treatment is applied to the governing equations, which leads to:

$$\frac{dJ_i}{dz} = 0|_r \quad (8)$$

$$-D_i \frac{d^2 C_i}{dz^2} - \frac{D_i z_i F}{RT} \frac{d}{dz} \left(C_i \frac{d\phi}{dz} \right) = 0|_r \quad (9)$$

$$\epsilon_0 \epsilon_r \frac{d^2 \phi}{dz^2} = -F \sum z_i C_i|_r \quad (10)$$

The 1D Nernst-Planck-Poisson equations listed above are solved with the following boundary conditions:

$$\frac{d\phi}{dz}|_{z_0} = -\frac{\eta_0}{\lambda_s} \log_{10}\left(\frac{j}{j_0}\right), J_1|_{z_0} = 0, J_2|_{z_0} = k_{\text{OH}^-} \quad (11)$$

$$C_1|_{z_h} = ca, C_2|_{z_h} = cx \quad (12)$$

where the boundary conditions at $z = z_0$ comes from Eq. 2, Eq. 3 and Eq. 7 with a current density j given as an input, and the boundary conditions at $z = z_h$ are iterative boundary conditions updated by the variables from the COMSOL in the last iteration step. As we only need the electric displacement field at the interface to update the boundary condition of the upper subdomain, $\frac{d\phi}{dz}$ is treated as a whole and thus only one boundary condition related to $\frac{d\phi}{dz}$ is needed. Also, since H_2 's concentration is not coupled with the electric field due to electric neutrality, it is not included in the MATLAB solver during the iteration process for simplification. When the convergence is achieved, both ϕ and C_{H_2} will be taken into consideration for the full-field solution.

For the upper half of the computational domain from the bottom of the Nernst diffusion layer to the bulk, a 2D axisymmetric, steady-state study of the Nernst-Planck-Poisson equation is performed in COMSOL Multiphysics, which uses finite element method (FEM) to discretize the PDEs into a system of nonlinear algebraic equations. This nonlinear system is then linearized using the Newton-Raphson method at each iteration to form a linear system that linear solvers can solve.² The bulk electrical potential ($\phi = 0$) and the bulk concentrations of the three species ($C_{\text{K}^+} = 0.1\text{M}, C_{\text{OH}^-} = 0.1\text{M}, C_{\text{H}_2} = 0.8\text{mM}$) are used as the initial guess. And it is worth noting that the MATLAB solver for the lower subdomain is coupled into the computation process as a boundary condition as follows

$$D_{z=z_h}^c = D_{z=z_h}^m = M(ca_{z=z_h}^c, cx_{z=z_h}^c) \quad (13)$$

¹Theoretically, the location of this domain decomposition interface can be changed, as long as the quasi-1D approximation holds for the lower subdomain. Also, increasing the thickness of the upper subdomain, the computational cost can be further reduced, yet at the cost of sacrificing accuracy. See Sec. 2.3 for details about this domain decomposition position.

²The iterations here can be automatically realized by the default setting of the COMSOL software. Here we clarify the logic behind as a reference for users who might want to realize the coupling logic in other softwares or open-source frameworks.

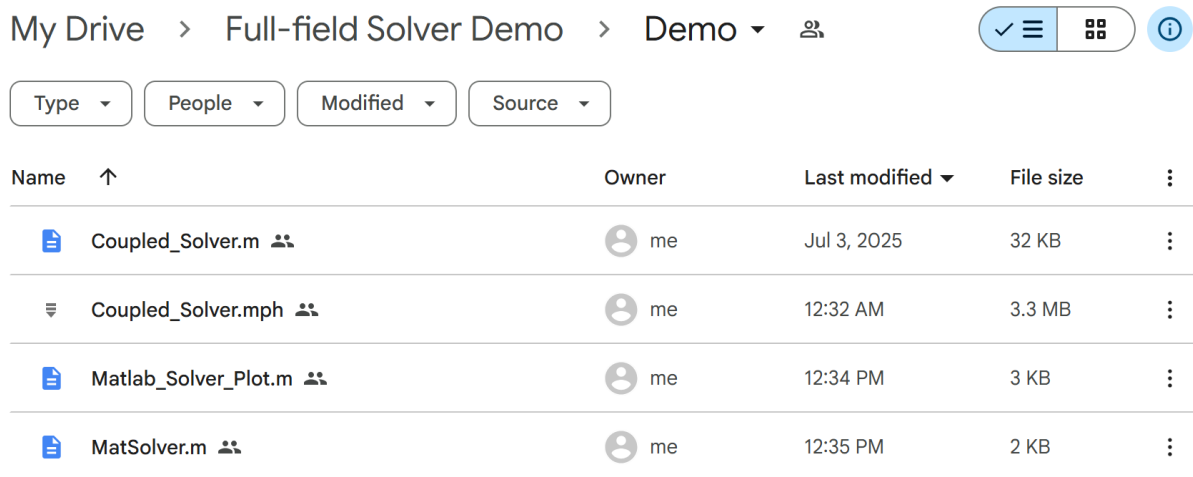
where D is the electric displacement field, ca, cx are cation and anion concentration respectively. The M denotes the MATLAB function that uses the boundary concentration as the input to output the boundary displacement field, and the superscripts c and m stand for the solution from COMSOL (the upper subdomain) and MATLAB (the lower subdomain), respectively. This boundary condition will be updated at each iteration step of the Newton-Raphson method. In this way, a bidirectional coupling is created, as the local concentrations affect the electric field via the boundary displacement field, and the electric field in turn influences species transport via migration terms. Such iteration is repeated until convergence is achieved in the COMSOL solver. Afterwards, the boundary concentrations from the converged solution are used as the input of the MATLAB solver to get the solution in the EDL, which is then combined with the solution in COMSOL to form the full-field solution.

2 Demonstration of Use

In this section, we demonstrate the use of the aforementioned computational framework. The governing equations are already explained in Model Formulation, and for the geometrical configuration, we take a 2D-axisymmetric case with a circular cathode in the middle as an example. The solver's logic can either be realized in the COMSOL Multiphysics graphic interface or based on MATLAB code. Here a step-by-step guide is provided based on the graphic interface followed by the introduction of the coupling based on MATLAB code and the matching conditions.

2.1 Step-by-step guide for using the solver

For the supporting files of this part, please see the Google Drive here. The content table for the provided link is shown in Fig.2. The following subsections will walk the reader through the steps needed to build the solver based on `Coupled_Solver.mph` and `MatSolver.m`, and We will further introduce the use of `Coupled_Solver.m` and `Matlab_Solver_Plot.m` later as another way to realize the coupling logic. It is recommended to refer to the corresponding file when reading each part, and the 4 files should be downloaded and stored in the same folder. For beginners, COMSOL's official user's guide on the Electrochemistry Module [4] is also a good resource to refer to.



Name	Owner	Last modified	File size
<code>Coupled_Solver.m</code>	me	Jul 3, 2025	32 KB
<code>Coupled_Solver.mph</code>	me	12:32 AM	3.3 MB
<code>Matlab_Solver_Plot.m</code>	me	12:34 PM	3 KB
<code>MatSolver.m</code>	me	12:35 PM	2 KB

Figure 2: Supporting files in the provided Google Drive link.

2.1.1 Setup of the COMSOL with MATLAB Livelink

For first-time users of the COMSOL with MATLAB Livelink, several steps need to be taken to ensure the smooth cooperation between COMSOL and MATLAB. When the COMSOL is properly installed together with MATLAB, first-time users need to go to the Preferences setting in the COMSOL graphic interface, for registering MATLAB as a COMSOL server and allowing the use of external MATLAB functions, as shown in Fig. 3 and Fig. 4.

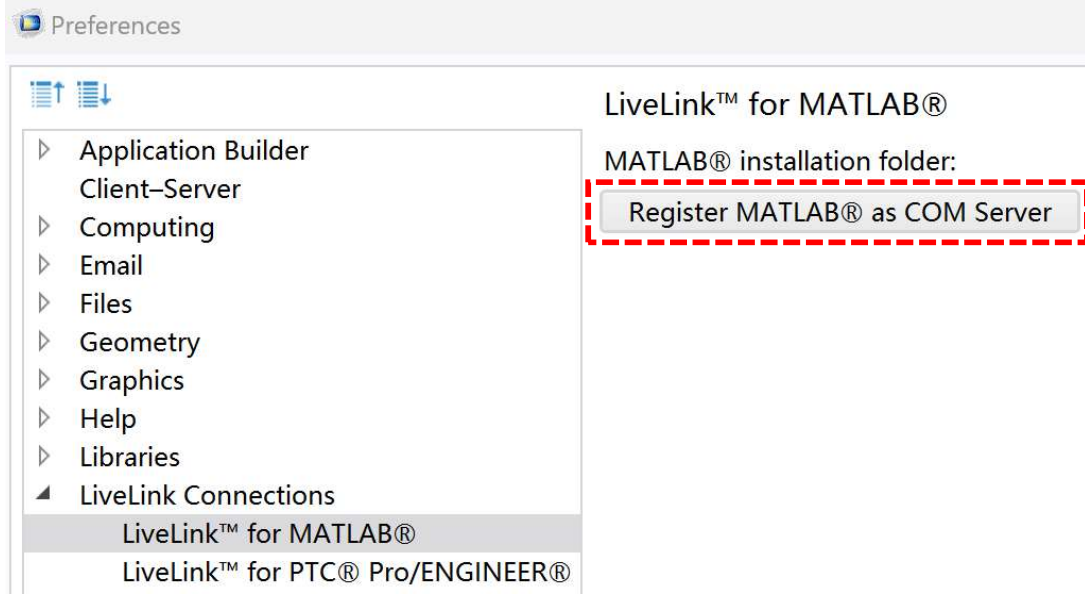


Figure 3: The Nernst-Planck-Poisson interface in COMSOL. In the Electrostatics interface, we define the Electric Displacement Field boundary condition using the MATLAB function.

2.1.2 Building the quasi-1D solver in MATLAB

The numerical solution of the 1D Nernst-Planck-Poisson system is implemented in MATLAB using the boundary value problem solver `bvp4c`. The solver workflow consists of three main components: initialization, boundary value problem definition, and solution processing. The core function `MatSolver` accepts bulk cation (`ca1`) and anion (`cx`) concentration arrays as inputs and returns the displacement field D_h at $z = h$.

Solver Initialization The computational domain spans from the cathode surface ($z = 0$) to the domain decomposition position ($z = h = 20$ nm). Spatial discretization uses a uniform grid with $\Delta z = 0.05$ nm. The parameters and physical constants used can be found in Tab. 1, and may be modified according to the need. The initial guess for the solution vector employs a decaying electric

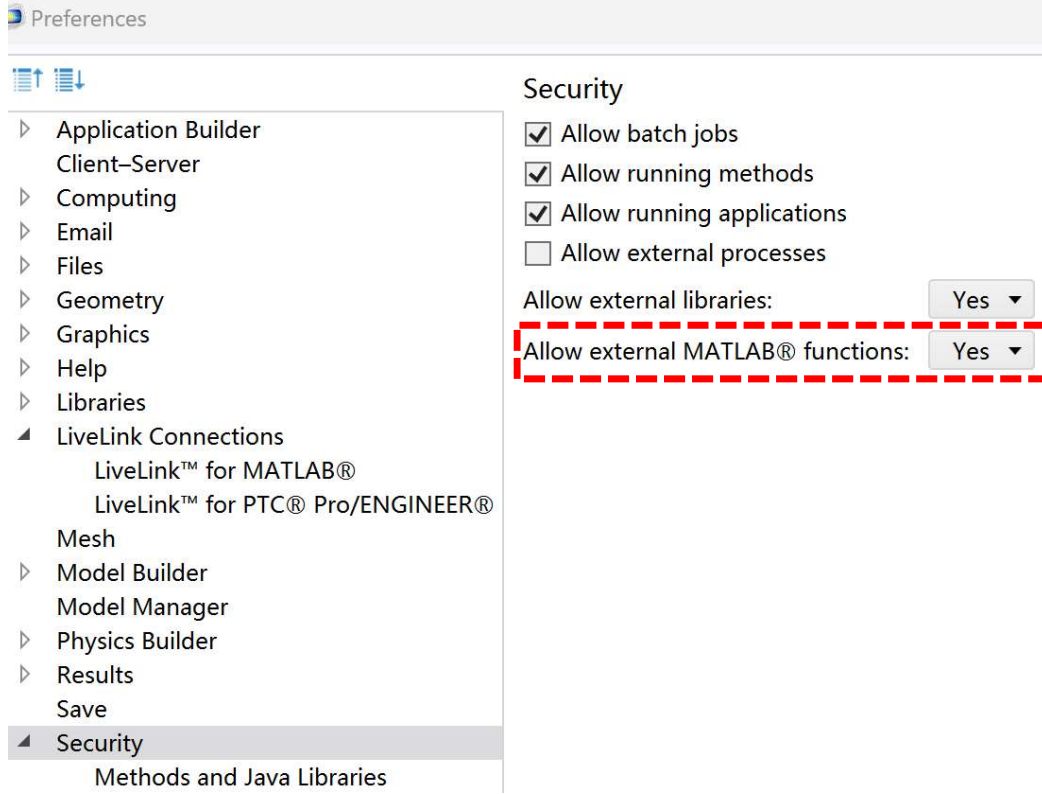


Figure 4: The Nernst-Planck-Poisson interface in COMSOL. In the Electrostatics interface, we define the Electric Displacement Field boundary condition using the MATLAB function.

field and linear anion concentration profile:

$$\text{guess}(x) = \begin{bmatrix} -1/(x+1) \\ 1/(x+1) \\ \mathbf{cx} \cdot x/h \end{bmatrix}$$

Boundary Value Problem Specification The coupled ODE system is defined in `bvpfcn` with state variables scaled for numerical stability:

1. Scaled electric field: $y_1 = E/10^6$ (V/ μm)
2. Scaled cation concentration: $y_2 = [\text{K}^+]/10^3$ (kmol/ m^3)
3. Anion concentration: $y_3 = [\text{OH}^-]$ (mol/ m^3)

The governing equations are implemented as:

$$\begin{aligned} \frac{dy_1}{dz} &= \frac{F}{\epsilon \cdot 10^6} (y_2 \cdot 10^3 - y_3) \\ \frac{dy_2}{dz} &= \frac{F}{RT} y_2 y_1 \\ \frac{dy_3}{dz} &= -\frac{J}{D_X} + \frac{F}{RT} y_1 y_3 \cdot 10^{-3} \end{aligned}$$

Boundary conditions enforce the Stern layer field and bulk concentrations:

$$\mathbf{bcfcn} = \begin{cases} y_1(0) = E_0/10^6 \\ y_2(h) = \mathbf{ca1}/10^3 \\ y_3(h) = \mathbf{cx} \end{cases}$$

Solution Procedure For each input concentration pair $(\mathbf{ca1}_i, \mathbf{cx}_i)$:

1. Update bulk concentration boundary conditions
2. Solve BVP using `bvp4c` with specified mesh and convergence criteria
3. Extract electric field at $z = h$: $E_h = y_1(h) \times 10^6$ (V/m)
4. Compute displacement field: $D_h = \epsilon \cdot E_h$ (C/m²)

The solver stores D_h values for all concentration pairs in the output array `D_hmatrix`.

The solution vector scaling (10^3 and 10^6 factors) maintains consistent numerical conditioning. The anion transport equation explicitly incorporates the reaction flux term $J = i_s/F$ reflecting OH^- generation at the cathode. The cation equation omits a reaction term as K^+ acts only as a charge carrier.

2.1.3 Building the 3D solver in COMSOL

With the quasi-1D solver completed, we now demonstrate implementation of the upper subdomain solver in COMSOL Multiphysics. The workflow comprises:

- Selecting the physics interface and defining the model
- Specifying parameters and variables
- Constructing the geometry and assigning materials
- Applying boundary conditions
- Generating the mesh and solving

Selecting the physics interface and defining the model

To start with, we first select modules needed from the options provided by the COMSOL software and clarify the study type, as shown in Fig 5. For convenience, here we leverage the Model Wizard provided by the software. While the problem itself is 3-dimensional, we can just select "2D Axisymmetric" due to its symmetry, which can significantly save computational cost compared to using a 3D model. For the stationary Nernst-Planck-Poisson system we need to solve, we can directly search for the "Nernst-Planck-Poisson Equations" interface consisting of two coupled module "Electrostatics" and "Transport of Diluted Species", and choose "Stationary" in "Select study. With this steps done, we should be able to see the interface shown in Fig. 6.

Specifying parameters and variables

The COMSOL Multiphysics software allows us to define parameters and variables that can be later used to define the geometry and boundary conditions, which provides an easy handle to change the working conditions of the model. In this case we will only use parameters, and since the model is not that complicated, we can directly define the parameters in the "Global Definition". The expression of each parameter can be either a numerical value, or an expression including other

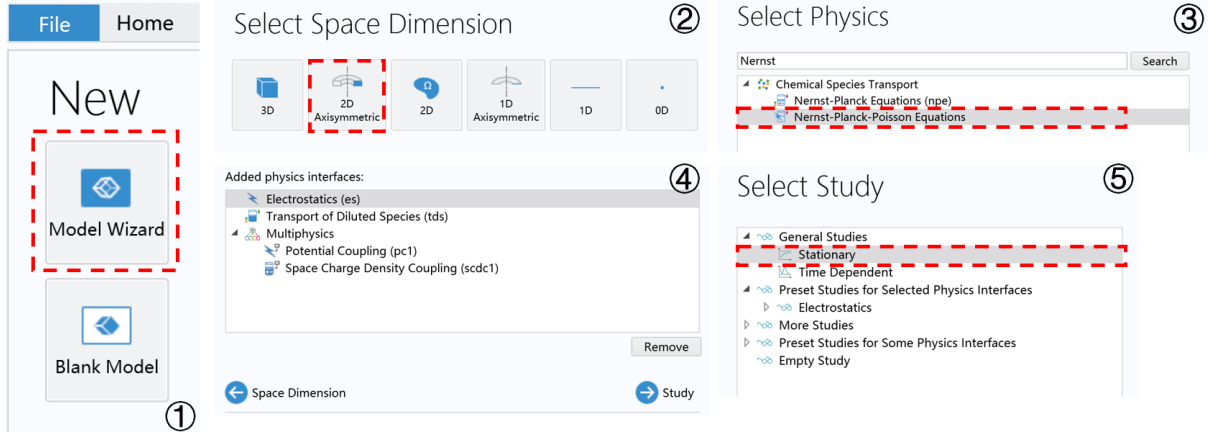


Figure 5: Steps to select the physics interface and define the model.

already-defined parameters. To add the parameters needed in our simulation case, the reader may refer to Tab. 1, or an easier way is just to open `Coupled_Solver.mph` and copy the parameter table from there.

It is worth noting that the quasi-1D solver in MATLAB solver will also be incorporated in the "Global Definition" part and serves a role similar to parameters and variables. The steps to add a MATLAB function in Global Definition are demonstrated in Fig 7. When setting the MATLAB function, the function name should strictly follow the function name in the MATLAB file for the quasi-1D solver, and the "Arguments" should be the variable name in the COMSOL part that will be used as the input to the MATLAB function.

Constructing the geometry and assigning materials

For the cylindrical system we are going to solve, since we have chosen 2D-axisymmetric for the space dimension, it will be represented by a rectangle. Therefore, the "Rectangle", together with "Points" and "Line segmentation" is enough for building the geometry in our problem, as can be seen in Fig 8.

As for the material, since the electrolyte is dilute solution, here the material properties of water are assigned to the entire region in COMSOL.

Applying boundary conditions

To get the right solution, it is of great significance to assign correct boundary conditions for both the "Electrostatics" module and the "Transport of Diluted Species" module. The first subfigure of Fig. 9 shows how we can find all the supporter boundary conditions in COMSOL for each module, which can also be found by right-click the corresponding module in the "Model Builder". The second subfigure of Fig. 9 emphasizes that we should use the MATLAB function to define the electric displacement field on the cathode. Other boundary conditions are easier to assign and can be found in the previous introduction about the model.

Generating mesh and solving

With the aforementioned steps done, the last important step is to discretize the geometry with a mesh, which is a fundamental requirement of the finite element method. While this can be done in different ways, a general concern is to use fine mesh near the cathode for accuracy and use slightly coarser mesh in the bulk region to save computational cost. In Fig. 10, triangular mesh is used for the entire geometry, with the area close to the cathode refined by using the "size" option. To further improve the resolution near the cathode, a boundary layer is also added. After

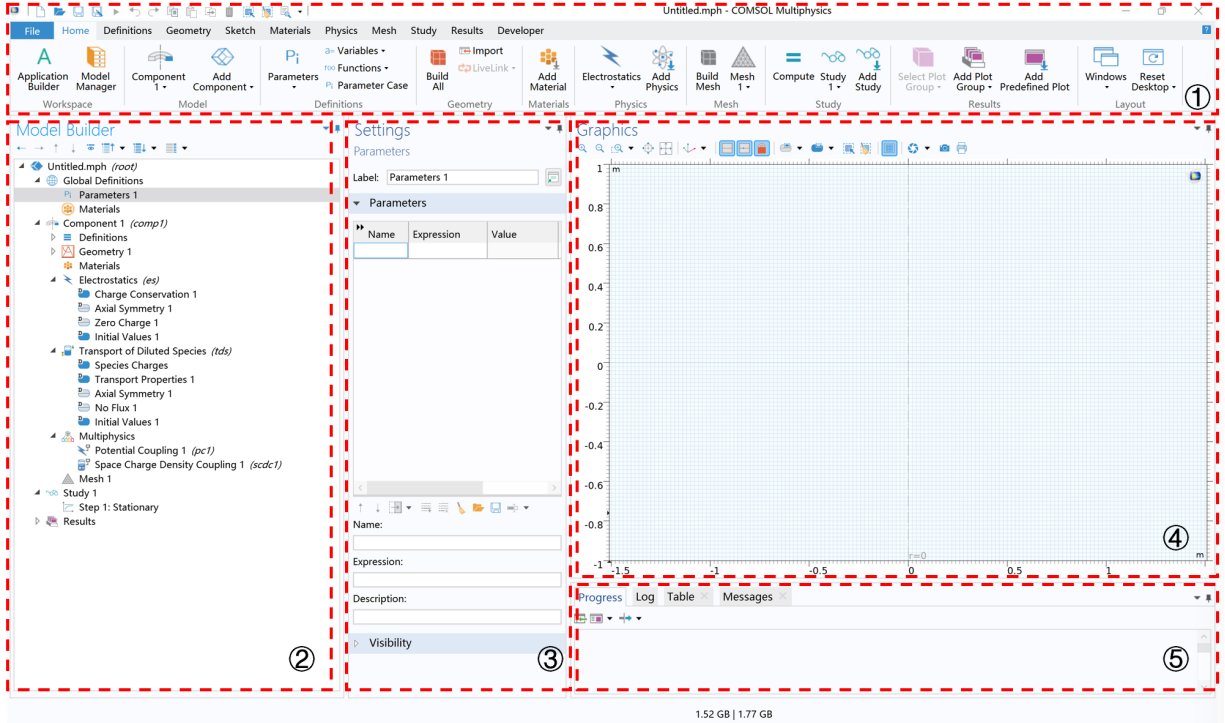


Figure 6: The graphic interface of COMSOL Multiphysics. the layout comprises five main sections: (1) the top Menu Bar for accessing core functions like File, Home, and Results tabs; (2) the left Model Builder tree organizing global definitions, components, geometry, physics, and study steps hierarchically; (3) the center-left Settings Window displaying editable inputs (e.g., Parameters/Variables tables with Name/Expression/Value columns); (4) the central Graphics Window for visualizing geometry, mesh, and solutions within the coordinate system; and (5) the bottom Status Bar showing real-time logs, messages, and solver diagnostics during model execution.

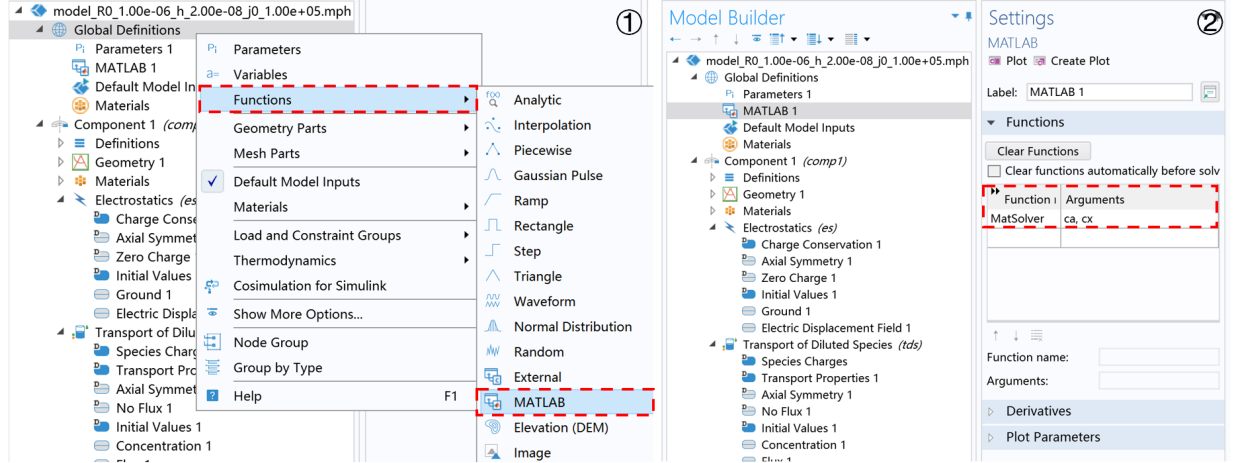


Figure 7: Steps to incorporate the quasi-1D solver in MATLAB with the 3D solver in COMSOL.

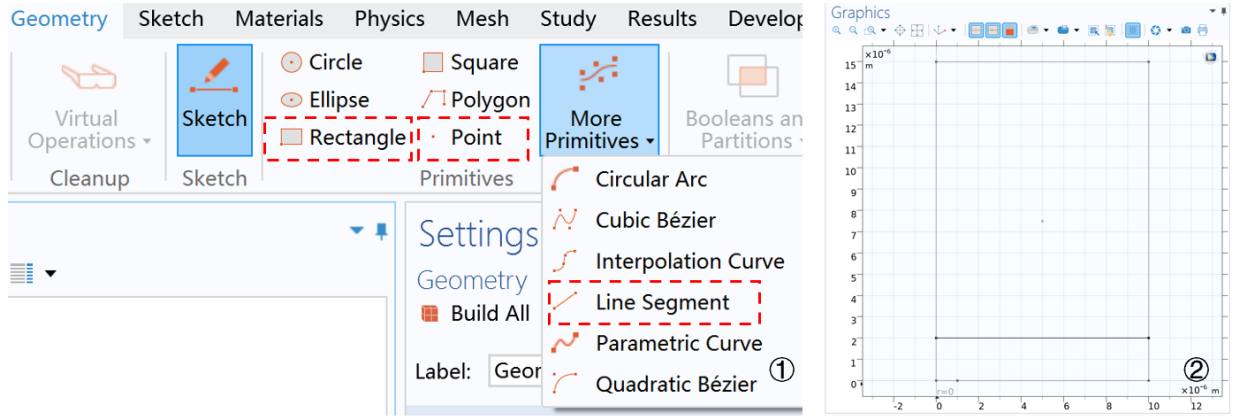


Figure 8: Steps to generate the geometry. (1) Geometry primitives needed to build the geometry in our case. (2) the generated geometry.

generating the mesh, it is recommended to check the mesh quality by using "Plot" or "Statistics", as demonstrated in Fig. 11. A quantitative review of the mesh quality will be given, with a value close to 0 indicating poor mesh and a value close to 1 indicating good mesh.

After a mesh of reasonable quality is generated, we can directly go to the "Study-Compute" in the top Menu Bar. This will call the default solver to numerically solve the problem, with the progress shown in the Status Bar.

2.2 The communication between COMSOL and MATLAB

2.2.1 Graphic Interface-based Coupling

The first way to realize the coupling logic is based on the graphic interface of COMSOL Multi-physics, which has been elucidated in Sec. 2.1. The major steps responsible for this coupling is the definition of the MATLAB function in "Global Definition" and the use of this function when specifying boundary conditions. Here we again highlight its role as the first pathway to realize the coupling logic.

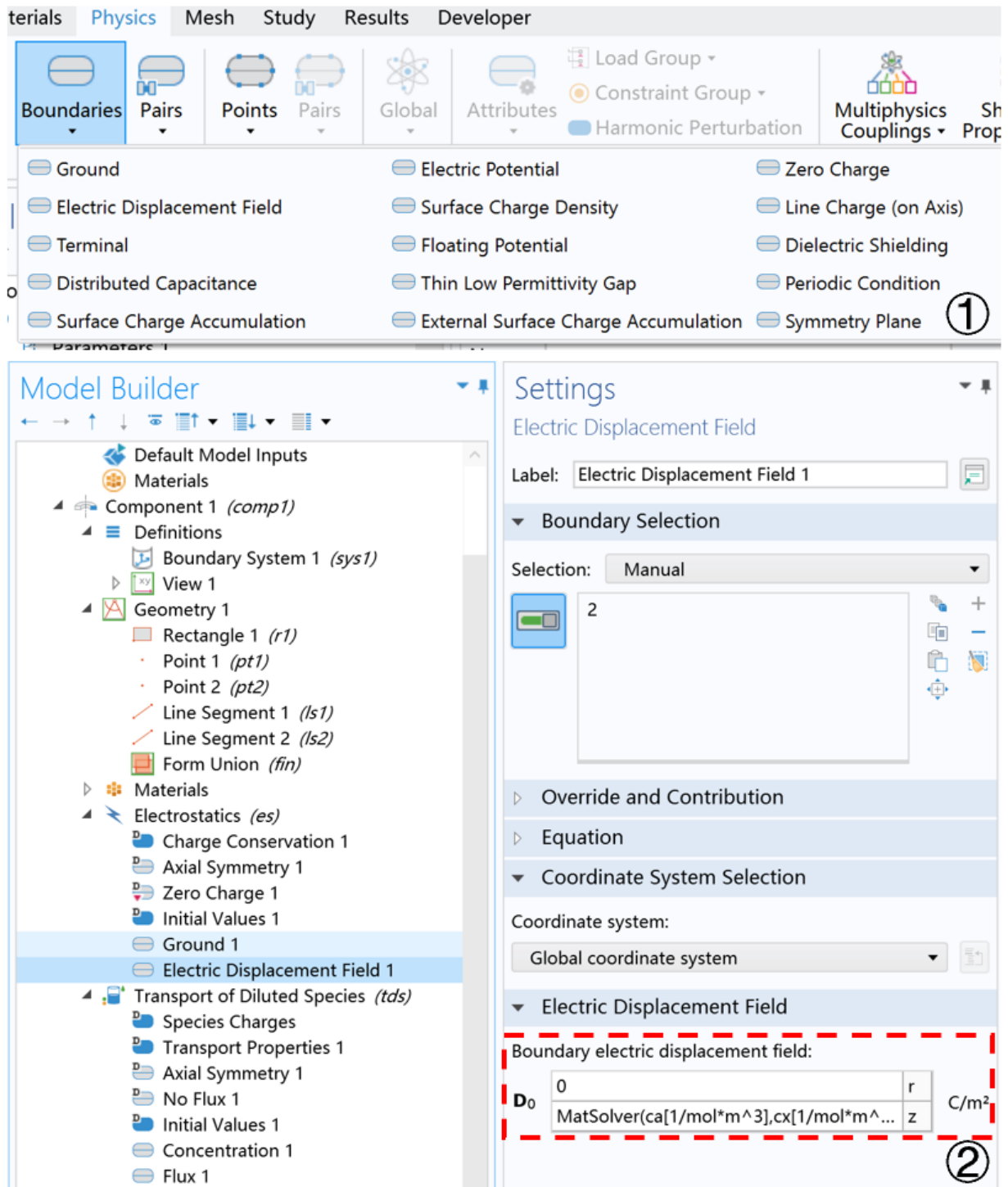


Figure 9: Steps to define boundary conditions. (1) The place to find the options for boundary conditions. (2) The way to use the MATLAB function as a boundary condition.

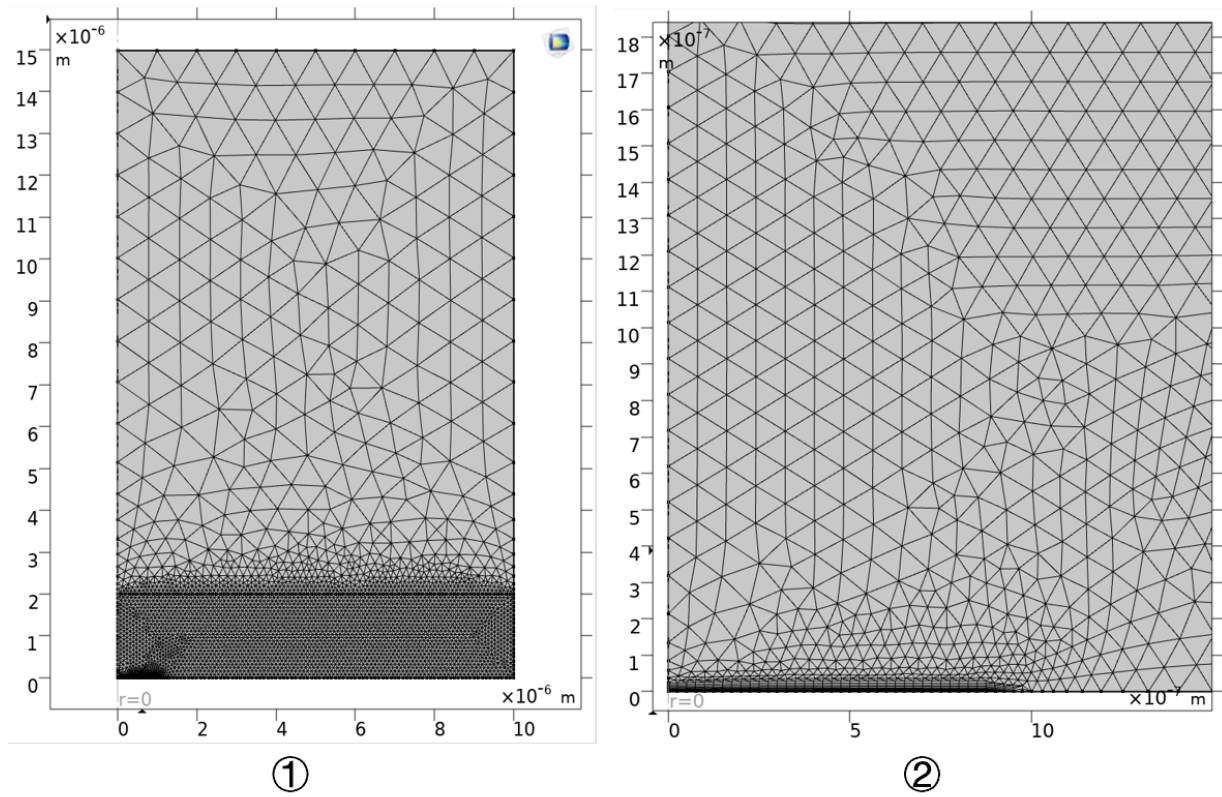


Figure 10: The generated mesh. (1) The mesh for the entire domain in COMSOL. (2) The mesh near the cathode.

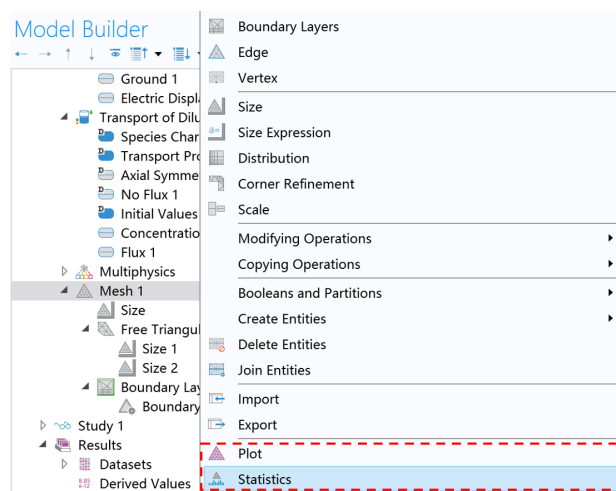


Figure 11: Options to check the mesh quality.

2.2.2 Code-based Coupling

The aforementioned computational framework can be implemented entirely using MATLAB code as well, as each operation in COMSOL Multiphysics translates into multiple lines of MATLAB code. As the underlying coupling logic remains unchanged when we turn to MATLAB for the simulation of the upper subdomain, in this subsection we focus on the conversion of the .mph file and .m file. This can be useful to offer flexibility for more complicated logic.

- **MATLAB code generation and preparation**

To replace the graphic interface-based calculation with MATLAB code, two approaches can be used. For simple cases, one can directly write the code corresponding to the operations in the COMSOL Multiphysics graphic interface.

- *Preparation of the MATLAB code for the upper subdomain:* To get the code for the upper subdomain, there are two approaches. For relatively simple cases, it can be an option to directly write the MATLAB code corresponding to the settings and operations in the COMSOL graphic interface. For more complicated cases, it would be easier to set up the case in COMSOL and convert it to the MATLAB code. Before saving it as a .m file, "Compact History" in the "File" menu should be performed to remove redundant steps that may appear in the converted MATLAB code. In the supporting files provided in Sec. 2.1, the main part of `Coupled_Solver.m` is just the converted MATLAB code from `Coupled_Solver.mph`.
- *Storage location:* Similarly, now that we have two MATLAB files for the upper and lower subdomain, respectively, they should be saved under the same directory so that access to each other will not be a problem.

- **Post-processing and visualization**

- *Post-processing:* After running the simulation case in COMSOL Multiphysics with MATLAB, post-processing can directly be done in MATLAB by using functions such as "mphinterp". More details of these functions can be found in Chapter 6 of the User Guide of COMSOL Livelink with MATLAB [5].
- *Visualization based on COMSOL graphic interface:* Though the simulation is run in MATLAB, we can still leverage COMSOL's graphic interface to visualize the results in a convenient way. A new COMSOL window should be opened and then connected to the server. This can be followed by importing applications from server, so that we can have the visualization in COMSOL graphic interface of a simulation case run in MATLAB, as shown in Fig. 12

2.3 Matching of the quasi-1D solution and the 3D solution

While the proposed computational framework can and should lead to a perfectly matched solution at the domain decomposition interface, it requires fine tuning to avoid or at least suppress numerical errors that can cause inconsistency. Besides continuity at the domain decomposition interface, another important criterion to check the precision of the solution is that the solutions should be independent of the domain decomposition position. In this subsection, we leverage a single-electrode case ($R_0 = 1\mu m, j = 10^5 A/m^2$) to highlight several key points for solver settings. The hydroxide ion concentration profiles are shown in Fig. 13, which is the combined effect of migration and diffusion and will be further discussed in the upcoming work.

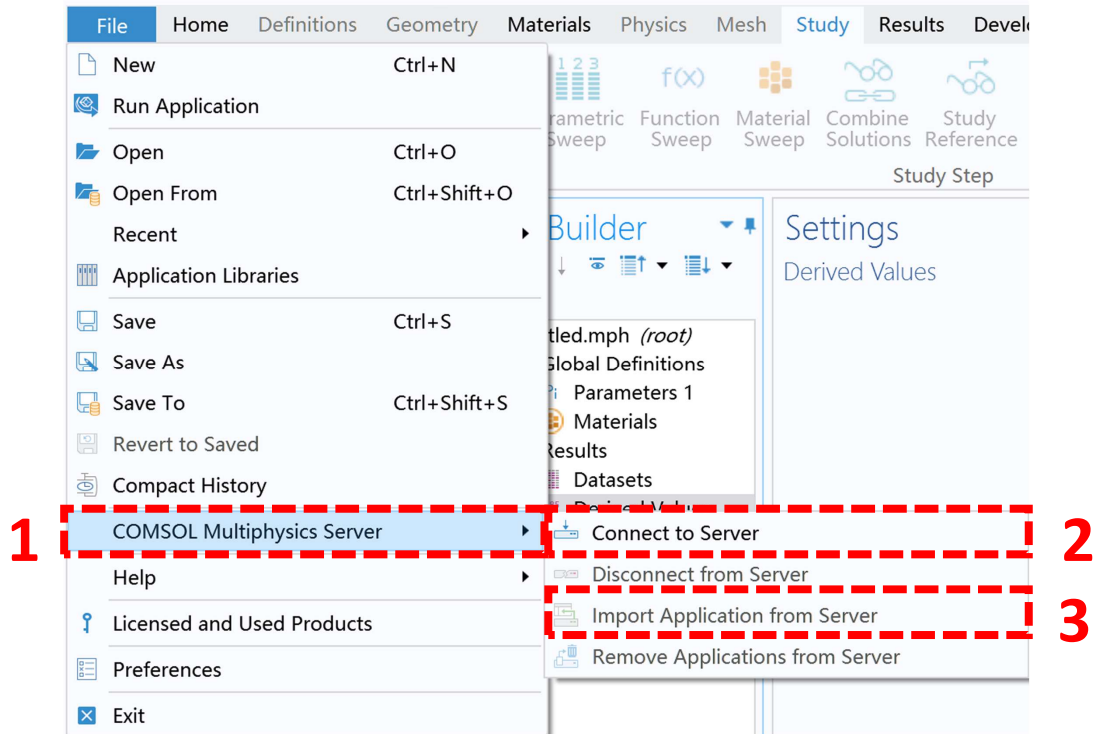


Figure 12: Steps to visualize the code-based simulation results in COMSOL.

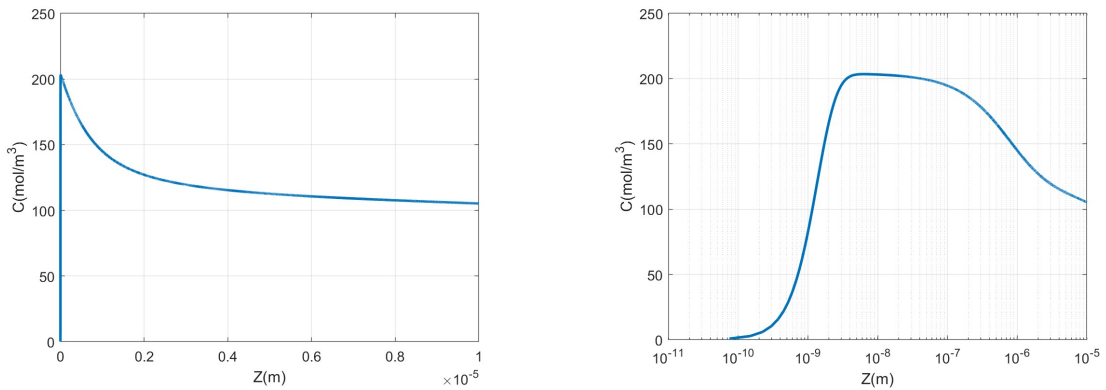


Figure 13: The concentration profile of OH^- in the z direction. The left subgraph is in the linear scale, and the right subgraph is in the logarithmic scale.

Safe range for domain decomposition position

$z_h(\text{nm})$	$z_m(\text{nm})$	$C_m(\text{mol/m}^3)$	Relative Error (%)
2	2	202.281	0.563
5	5	202.278	0.565
10	6.4	203.427	0.000
20	6.4	203.427	0.000
30	6.4	204.445	0.500
40	6.3	205.420	0.979
50	6.3	206.394	1.458
100	6.25	211.270	3.855

Table 2: Solutions from different domain decomposition positions

In terms of capturing the important characteristics of the peak, especially the peak position z_m and the peak concentration C_m , there is a safe range of domain decomposition positions that can strike a best balance between accuracy and computational cost. With the other settings fixed, we vary the domain decomposition position z_h from 2nm to 100nm , and the observed results are shown in Tab. 2. By comparing the results, we can see that under the current setting, the peak concentration occurs at approximately 6.4nm . When the domain decomposition position z_h is under this physical peak, the solver fails to capture the exact peak position but instead returns a value same as z_h . This is due to the difference of mesh quality across the domain decomposition interface. While the mesh size in the quasi-1D subdomain can be at the scale of 0.01nm or even smaller, the mesh size in the 3D subdomain can quickly grows to 100nm or 100nm depending on the system size. The mesh in the 3D domain is too coarse to resolve the peak, and it can lead to remarkably increased computational cost when further refined. Therefore, to strike a balance between accuracy and computational cost, it is recommended to place the physical peak in the quasi-1D domain, i.e., $z_h > z_m$. Also, with $z_h > z_m$, it is observed that C_m grows with z_m . This is because the mass transport is inherently 3D, but the radial component is fixed as 0 in the area under the domain decomposition interface. Therefore, the bigger the z_h , the bigger the mass flux component in the z direction. If we consider the mass transport at $z > z_h$, which is mainly governed by diffusion, the increased mass flux component in the z direction due to quasi-1D treatment results in an increased C_m . With the abovementioned observations, it is recommended to decompose the domain at 2 to 3 times of z_m .

Compatibility of mesh size across the boundary

Consistent with previous discussion, the mesh resolution in the 3D subdomain is intentionally coarser compared to the quasi-1D subdomain, achieving computational efficiency without significant loss of accuracy. However, the selection of mesh sizes on two sides should not be random, as the mismatch of the mesh sizes can hinder the smooth transition of the to-be-solved variables across the boundary. Take the full-field concentration profile as an example, typical numerical errors due to this mismatch include kinks at the domain decomposition position and oscillation of the concentration profile in the quasi-1D subdomain.³ Based on previous trials, a safe upper limit for the ratio of the mesh size in the z direction in the 3D subdomain and in the quasi-1D subdomain is approximately 2000. If higher resolution in the quasi-1D subdomain is needed, the user might consider replacing the uniform mesh in the quasi-1D subdomain with a non-uniform mesh, the size of which grows when approaching the domain decomposition interface.

³The author is trying to rebuild the ill meshes that lead to these problems, and the corresponding meshes will be updated in the Google Drive link.

References

- [1] Muhan Zhang, Weihong Li, and Lenan Zhang. A unified multiscale framework for simulating electrochemical gas evolving reactions. In *Electrochemical Society Meeting Abstracts 245*, number 45, pages 2494–2494. The Electrochemical Society, Inc., 2024.
- [2] Kai Zeng and Dongke Zhang. Recent progress in alkaline water electrolysis for hydrogen production and applications. *Progress in energy and combustion science*, 36(3):307–326, 2010.
- [3] Congling Hu, Lei Zhang, and Jinlong Gong. Recent progress made in the mechanism comprehension and design of electrocatalysts for alkaline water splitting. *Energy & Environmental Science*, 12(9):2620–2645, 2019.
- [4] COMSOL. *Electrochemistry Module User’s Guide*. COMSOL AB, 2022. Accessed: [2025-03-21].
- [5] COMSOL. *LiveLink for MATLAB User’s Guide*. COMSOL AB, 2022. Accessed: [2025-03-21].